

Osmium Components

Code and Documentation by Avery Norris

Components are the main change in Osmium's take on a modern, unbiased **GameObject-Component system** or **EC/ECS** ([Entity Component System](#))[↗]. They vary often, but the most popular systems such as **Unity**, **Godot** or **Unreal** traditionally involve 2/3 types of data.

Parent Scene/**World** :

Children GameObjects/**Actors** :

Children Components/**Scripts**

- **Scenes** are basically the worlds these objects inhabit. Common *synonyms* are Worlds or Levels. If GameObjects are files in our analogy, then scenes are separate hard drives. The scene's jobs mainly center around being a Docker. (*Rather than an object of functionality*)
- **GameObjects** are the parents components are glued to, in most Game Libraries and Engines they are also treated as physical objects; and often given properties like **Transform** and **Tags**. GameObjects can also commonly nest in each other. I sometimes find it useful to view these as Directories for components.
- **Components** are what we actually care about, shockingly, they are the "Component" in Entity Component System, and you can think about them as the actual scripts or program in your Game, but a little more object-oriented.

Keep in mind that this is a general description of **other** Game Development Platforms. It's impossible to summarize every single **ECS** in the world. (A lot of ECS' also use **systems** alongside this hierarchy)

How Osmium Differs

As of this current version, Osmium's **ECS** has taken on a different form than most.

Parent Scene/World

Components :

Children Components

One of the main **Osmium Principles** is **Generalization**; and during development it became clear **GameObjects** are unnecessary, which lead to the **Component** partially taking over the job.

Rather than the traditional **ECS** **Components** (Such as Unity), Osmium's **Components** are actually closer to a combination of the **GameObjects** and **Components** we discussed earlier.

Osmium does not implement many fancy features out of the box. **Bias** is almost a requirement for **GameObjects** to be useful. But in Osmium, where even position isn't implemented by default, **GameObjects** tend to become basically useless objects; that just hold **Components** and transfer **Events**

Because of this I decided to make a more flexible type of entity that can act as **GameObject** and **Component** at once. However, **Components** still do not implement many features out of the box, instead we use their **expandability** to our advantage. (Which will be explained later)

How To Use Events

Components are rather easy to control, similar to Engines like Unity, you can make your own custom script by inheriting the abstract "**Component**" class.

```
public class MyScript : Component {}
```

On the surface level, **Components** provide you with your current scene and parents.

They also give very handy Game Events which are present in 99% of Game Development Platforms.

Load

Unload

Update

Draw

Create

Remove

- Load and Unload provides a call when the Game is opened and closed respectively, please be wary: these will not call under a force close.
- Update and Draw both trigger each frame, starting with Update then Draw, like most frameworks. It is recommended to put your drawing code within Draw() and keep almost everything else inside Update()

- Finally Create and Remove are called when the Component is spawned In/Out. It should be noted that Create is called **after** the constructor. If you try to make certain references or calls in the constructor it may not fully work, as the object is half instantiated. It is recommended to use Create when possible. Also, Remove will not be called if the program is closed, only if it is actually called with Remove(component)

You can call these events by adding them to your **Component-Inheriting-Class** as shown below. And **Osmium** will automatically recognize it as an event! ([If you want to see under the hood](#))

```
public void Update() {}
```

Putting this code inside any Component inheriting class, will create a method that gets called every single frame, just like that!

Important Information

There are a few random tidbits that also make Osmium angry:

- Components cannot have constructor Parameters, while **Empty** constructors do work, It's recommended to just use Create()
- Some properties of the Component are calculated and NOT stored as one would expect. For instance: Scenes are not actually stored in the class, and they are calculated each time you use them. This is very inexpensive, but you should keep it in mind while developing; and cache when needed.
- There are many informative attributes tied to members of **Component**; I would recommend you look at them before using any tool inside Component. For instance: there are marking attributes such as CalculatedProperty that inform you of a member that is not stored but calculated each time you use it.
- A good chunk of the Component's functionality is actually coming from the Docker! If you cannot find the Documentation for something, it is probably actually a Docker related object.

Examples and Good Practice

Since **Osmium** Components are slightly different compared to other ECSs; I thought it would be fitting to create a few examples of good practice or "tactics" you can do.

First let's see how we can recreate typical GameObject behavior; I would most recommend using **Nested Components** to achieve this. If we pretend we have implemented a few modules for basic transform profiles and sprite management, then we can easily make a basic movable sprite object. Like so :

Parent Scene/World

Empty Component :

Transform Component

Sprite Component

We can expand upon this easily as well. Say we want to make it into a moveable player character, we can modify the Empty Component to carry some additional functionality.

Parent Scene/World

Player Controller Component : <--

Transform Component

Sprite Component

If we want to give it a hitbox.

Parent Scene/World

Player Controller Component :

Transform Component

Sprite Component

Hitbox Component <--

And maybe let's say we want to scale or offset that

Parent Scene/World

Player Controller Component :

Transform Component

Sprite Component

Hitbox Component

Transform **Component** <---

Of course, there is some additional programming that would be needed between each step. (**Ex. Hitboxes listening to the child transform**), but you can see how this data structure builds intuitively.

It can be handy to compartmentalize any repeating pieces of code or types into a **Component**. It is also not immediately obvious at first, But I would say one of the largest utilities from an **object free component** system is the ability to function at a high level in the scene.

Often times in component-object **ECS**' you will have a **singleton** object/s that stores important one off Components, such as the *Camera*, or the *Game's Asset Loader*. By virtue of GameObjects, these **singletons** are required to have at least an empty GameObject as a parent. I've always found this to be unsatisfying. Luckily, Osmium Components can operate as standalone objects! You can insert a standalone Camera Component into the scene. Which feels cleaner and makes much more logical sense.

Parent Scene/World

Camera

Asset Loader

Under the Hood

As you've seen part of what makes **Components** so great is the fact that they can nest within themselves infinitely. This is possible because of an essential piece known as the **Docker**.

Dockers are seen everywhere in Osmium. They are built to store child Components. The Component class carries a Docker like so.

Component : **ComponentDocker**

Dockers also provide the Add, Get and Destroy functions we all know and love, along with the list of child Components. It is also responsible for Osmium Events being passed through the Scene, for Example, an Update call would look like this

```
Scene    -> Docker                                -> Component(Docker's child)
Update() -> ChainEvent(EventType.Update) -> Update()
```

For anyone wondering why **ComponentDocker** is not a part of Component, or any other details: Please see ([ComponentDocker.md](#))

Specialized Components

Because of its focus on **Expandability**, Osmium also allows **third-party** Components to enter the Scene. This allows for specialized Component or streamlined development, if you are willing to make assumptions.

Let's imagine you are making an enemy for an RPG and your Component Layout looks somewhat like this

Parent Scene/World

Enemy Pathfinding Component :

Transform Component

Sprite Component

Hitbox Component

Health Component

UNIQUE Component

In this diagram, **UNIQUE Component** is just a placeholder for any future or enemy specific Components, and it is critical to our example. And let's imagine that any time you add an enemy, then you will probably be adding some sort of "UNIQUE Component" there; And this unique Component often uses aspects from the others present.

In a scenario like this, it would likely behoove you to create a specialized Component type. As mentioned earlier, Components are identified by inheriting the abstract Component class. And it is possible to build an in-between class for additional functionality!

- Traditional Pattern

Your Script : Component : Docker

- Example Specialized Pattern

Your Script : EnemyComponent : Component : Docker

As you can see we inherited through a new (Theoretical) class called EnemyComponent rather than Component. This is perfectly legal; and intended! You can do virtually anything in your specialized Component in-between. I most recommend making use of **lambda** in situations like this.

For instance, you can provide simpler access to another Component like so

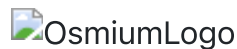
```
int Health -> Parent.Get<Health>().Health;
```

Any future EnemyComponents can simply put "**Health**" and it will correctly retrieve the dynamic value.

I should mention that this power comes with **great responsibility**. In this case: using EnemyComponent without a Health Component will cause logged errors, and possibly a runtime error/halt. But this functionality poses to harm to anyone using **Components** as normal.

End Of Documentation

Code and Documentation by Avery Norris



Component Docker

Code and Documentation by Avery Norris

One major and necessary requirement of an **EC/ECS** ([Entity Component System](#))[↗] is **Nested-Components**; In **Osmium's** own **ECS** we have two objects that need to be able to hold **Children-Components**

Scenes

Components

Originally, both classes carried methods to store **Components** as children; However this was extremely redundant, and **Component-Docker** was created to handle this in an organised fashion!

Purpose

Called a **Docker** for short; the **Component-Docker** carries **three main functionalities**

Storing Children Component

Providing [an](#) Interface [to](#) Modify Children

Transferring Events [to the](#) appropriate Components

- **Storing Children** is the **Main Job** of a Docker. Both Scenes and Components can host children. And the docker inheritance is what accommodates this. It uses a HashSet internally, and hashes Components in a list and two Dictionaries based on Priority, Type and Tags
- Osmium Provides an **Extensive Interface** to work with Components. You can check the API to see the MANY overloads for adding, getting, removing and moving Components provided.
- **Transferring Events** is the final usage for the **Docker**. When an Event is sent in **Osmium**, it is passed down between Dockers via ChainEvent(). Dockers call all the Dockers below themselves, and their inherited type. This is stopped if a Docker is a Component that is not enabled, or we reach the leaves of all branches in the **Component-Tree**.

End Of Documentation

Code and Documentation by Avery Norris



Scene

In **Osmium**, The **Scene** is responsible for Containing all the "Action". It inherits a **ComponentDock**; and carries many Components.

In Osmium, they are treated as **Even Entities** and can be loaded in parallel like so :

Osmium

Loaded **Scene A**

Loaded **Scene B**

Both Scene A and B will receive events independent of each other. It is also good to note that Components cannot exist at the **Scene-Level** (parallel of Scenes, like how A is to B). Components can only exist as a child of a Scene or a Child of a Component in a Scene.

Usage

Using a Scene is easy! Simply call **Osmium.AddScene(string name)** to create a new Scene! Or you can add your own with **Osmium.AddScene(Scene scene)**. The reason you might want to add your own is to allow for custom scene types. If you have seen [Components](#), then you might know that Osmium **supports** external inheritance! It's as easy as inheriting Scene, and using AddScene() to make your own Custom Scene Code!

For further guidance, refer to the **API** documentation! With **Osmium** you are able to Add, Get, and Remove Scenes.

End Of Documentation

Code and Documentation by Avery Norris

